

```

# BLOB Principle - Protocole F - Mesures hierarchiques
# Caracterisation de la dynamique simplexe -> arete -> triangle -> tetraedre
# avec respect du vocabulaire : pas de "destruction", on parle de
# regression hierarchique, redistribution, transition descendante
#
# Vocabulaire adopte (cadre Rudolphe Mirante - Blob Principle) :
#   - Conservation simplexiale : N est invariant
#   - Mitose simplexiale : projet futur (Blob 2.0), inscrite [0] long terme
#   - Ascension hierarchique : creation de structure superieure
#   - Regression hierarchique : retour vers niveau inferieur
#   - Reorganisation : transition conservant le niveau global
#
# Trois mesures :
#   F1 - Conservation (simplexiale et viabilite)
#   F2 - Cycle de vie des aretes avec profil temporel d engagement (Option B)
#   F3 - Comptage des tetraedres et stratification des transitions

import sys
import os

# =====
# AUTO-TEST CORRUPTION
# =====

def auto_check():
    print("=" * 70)
    print("  AUTO-TEST CORRUPTION CARACTERES")
    print("=" * 70)
    try:
        if hasattr(sys, 'argv') and len(sys.argv) > 0 and os.path.exists(sys.argv
            source_file = sys.argv[0]
        else:
            source_file = __file__ if '__file__' in globals() else None
        if source_file and os.path.exists(source_file):
            with open(source_file, 'r', encoding='utf-8') as f:
                content = f.read()
            dangerous = {
                chr(8216): "U+2018", chr(8217): "U+2019",
                chr(8220): "U+201C", chr(8221): "U+201D",
                chr(8211): "U+2013", chr(8212): "U+2014",
            }
            found = [(name, content.count(c)) for c, name in dangerous.items() if
            if found:
                print("ERREUR caracteres corrompus :", found)
            return False

```

```

        star = chr(42)
        bad_init = star + star + "init" + star + star
        if content.count(bad_init) > 1:
            print("ERREUR : motif corrompu init")
            return False
        triple_back = chr(96) * 3
        if content.count(triple_back) > 1:
            print("ERREUR : Markdown")
            return False
        print("OK : pas de corruption")
    else:
        print("Note : auto-test partiel")
    except Exception as e:
        print("Note : " + str(e))
    print("=" * 70)
    return True

if not auto_check():
    sys.exit(1)

# =====
# IMPORTS
# =====
import numpy as np
import time
import json
import gc
from collections import defaultdict, Counter

# =====
# Parametres canoniques
# =====
KAPPA_0 = 8
BETA = 0.30
TAU = 0.50

OUTPUT_DIR = '/content/blob_F_results'
os.makedirs(OUTPUT_DIR, exist_ok=True)

# =====
# Classe BlobGraph etendue avec mesures hierarchiques
# =====
class BlobGraphHier:
    def __init__(self, N, seed, is_null=False):

```

```

self.N = N
self.is_null = is_null
self.rng = np.random.default_rng(seed)
self.E_target = int(N * KAPPA_0 / 2)

# Structure du graphe
self.adj = [set() for _ in range(N)]
self.edge_list = [] # liste indexee
self.edge_id_map = {} # (u,v) -> indice dans edge_list
self._next_edge_id = 0 # identifiant unique pour suivi temporel

# Suivi cycle de vie des aretes
# Chaque arete a un id unique. On enregistre :
# - pas de naissance
# - pas de mort (None si vivante)
# - profil d engagement temporel : liste de (pas, n_tri_communs)
self.edge_birth = {} # edge_id -> birth_step
self.edge_death = {} # edge_id -> death_step
self.edge_endpoints = {} # edge_id -> (u, v)
self.edge_engagement_profile = {} # edge_id -> liste [(step, n_tri_commu
self.edge_max_engagement = {} # edge_id -> max_n_tri_atteint

# Mapping arete actuelle -> id
self.edge_position_to_id = {} # idx dans edge_list -> edge_id

self._init_graph()
self.deg = np.array([len(self.adj[i]) for i in range(N)], dtype=np.int32)
self.n_tri = self._count_triangles()
self.n_tetra = self._count_tetrahedra()
self.S_curr = self._compute_S(self.n_tri)

# Compteurs de transitions (mesure F3)
self.transitions = Counter()

self._step_counter = 0

def _new_edge_id(self):
    eid = self._next_edge_id
    self._next_edge_id += 1
    return eid

def _add_edge_tracked(self, u, v, step):
    """Ajoute une arete avec suivi complet."""
    u, v = min(u, v), max(u, v)
    eid = self._new_edge_id()
    self.adj[u].add(v)
    self.adj[v].add(u)

```

```

        idx = len(self.edge_list)
        self.edge_list.append((u, v))
        self.edge_id_map[(u, v)] = eid
        self.edge_position_to_id[idx] = eid
        self.edge_birth[eid] = step
        self.edge_endpoints[eid] = (u, v)
        self.edge_engagement_profile[eid] = []
        self.edge_max_engagement[eid] = 0
        return eid

def _remove_edge_tracked(self, idx, step):
    """Retire arete par index, marque sa mort."""
    u, v = self.edge_list[idx]
    eid = self.edge_position_to_id[idx]
    self.adj[u].discard(v)
    self.adj[v].discard(u)
    self.edge_death[eid] = step
    del self.edge_id_map[(u, v)]

    # Swap-pop sur edge_list
    last_idx = len(self.edge_list) - 1
    if idx != last_idx:
        last_edge = self.edge_list[last_idx]
        last_eid = self.edge_position_to_id[last_idx]
        self.edge_list[idx] = last_edge
        self.edge_position_to_id[idx] = last_eid
    self.edge_list.pop()
    del self.edge_position_to_id[last_idx]

def _init_graph(self):
    # Anneau initial
    for i in range(self.N):
        u = i
        v = (i + 1) % self.N
        if v not in self.adj[u]:
            self._add_edge_tracked(u, v, 0)
    # Remplissage aleatoire
    attempts = 0
    max_a = self.E_target * 10
    while len(self.edge_list) < self.E_target and attempts < max_a:
        u = int(self.rng.integers(self.N))
        v = int(self.rng.integers(self.N))
        attempts += 1
        if u != v and v not in self.adj[u]:
            self._add_edge_tracked(u, v, 0)

def _count_triangles(self):

```

```

        return sum(len(self.adj[u] & self.adj[v]) for u, v in self.edge_list) //

def _count_tetrahedra(self):
    """
    Compte les 4-cliques (tetraedres).
    Algorithme : pour chaque arete (u,v), pour chaque w dans N(u) inter N(v),
    compter les pairs w,w' dans N(u) inter N(v) avec w-w' arete.
    Plus simple : iterer sur triangles et compter les sommets x adjacents
    aux 3 sommets du triangle.
    """
    n_tetra = 0
    seen_tetra = set()
    for u, v in self.edge_list:
        common = self.adj[u] & self.adj[v]
        common_list = sorted(common)
        for i, w in enumerate(common_list):
            for x in common_list[i+1:]:
                # Verifier si w-x est aussi une arete
                if x in self.adj[w]:
                    # Tetraedre {u,v,w,x}
                    tetra_key = tuple(sorted([u, v, w, x]))
                    if tetra_key not in seen_tetra:
                        seen_tetra.add(tetra_key)
                        n_tetra += 1
    return n_tetra

def _count_tetra_through_edge(self, u, v):
    """
    Nombre de tetraedres passant par l arete (u,v).
    = nombre de paires (w,x) avec w,x dans N(u) inter N(v) et w-x arete.
    """
    common = self.adj[u] & self.adj[v]
    if len(common) < 2:
        return 0
    common_list = list(common)
    count = 0
    for i, w in enumerate(common_list):
        for x in common_list[i+1:]:
            if x in self.adj[w]:
                count += 1
    return count

def _compute_S(self, n_tri_val):
    mu = float(self.deg.mean())
    D = float(self.deg.std()) / mu if mu > 0 else 0.0
    return np.log(1 + n_tri_val) - BETA * D

```

```

def step_blob(self, current_step):
    """Pas Blob avec tracking complet."""
    self._step_counter = current_step
    idx = self.rng.integers(len(self.edge_list))
    u_rm, v_rm = self.edge_list[idx]
    u_add = int(self.rng.integers(self.N))
    v_add = int(self.rng.integers(self.N))
    if u_add == v_add or v_add in self.adj[u_add]:
        return False, None
    u_add, v_add = min(u_add, v_rm), max(u_add, v_rm)
    if (u_add, v_add) == (u_rm, v_rm):
        return False, None

    # Calcul des Delta avant modification
    dt_rm = len(self.adj[u_rm] & self.adj[v_rm])
    dtetra_rm = self._count_tetra_through_edge(u_rm, v_rm)

    # Retirer temporairement
    self.adj[u_rm].discard(v_rm)
    self.adj[v_rm].discard(u_rm)
    self.deg[u_rm] -= 1
    self.deg[v_rm] -= 1

    dt_add = len(self.adj[u_add] & self.adj[v_add])
    dtetra_add = self._count_tetra_through_edge(u_add, v_add)

    # Ajouter temporairement pour calcul S
    self.adj[u_add].add(v_add)
    self.adj[v_add].add(u_add)
    self.deg[u_add] += 1
    self.deg[v_add] += 1

    new_n_tri = self.n_tri - dt_rm + dt_add
    if new_n_tri < 0:
        # Annulation
        self.adj[u_add].discard(v_add)
        self.adj[v_add].discard(u_add)
        self.deg[u_add] -= 1
        self.deg[v_add] -= 1
        self.adj[u_rm].add(v_rm)
        self.adj[v_rm].add(u_rm)
        self.deg[u_rm] += 1
        self.deg[v_rm] += 1
        return False, None

    new_S = self._compute_S(new_n_tri)
    log_alpha = self.N * (new_S - self.S_curr) / TAU

```

```

if log_alpha >= 0:
    accepted = True
else:
    accepted = self.rng.random() < np.exp(max(log_alpha, -500))

if not accepted:
    # Annulation
    self.adj[u_add].discard(v_add)
    self.adj[v_add].discard(u_add)
    self.deg[u_add] -= 1
    self.deg[v_add] -= 1
    self.adj[u_rm].add(v_rm)
    self.adj[v_rm].add(u_rm)
    self.deg[u_rm] += 1
    self.deg[v_rm] += 1
    return False, None

# Acceptation effective : on doit annuler l ajout temporaire
# et faire les modifications avec tracking propre
self.adj[u_add].discard(v_add)
self.adj[v_add].discard(u_add)
self.deg[u_add] -= 1
self.deg[v_add] -= 1
self.adj[u_rm].add(v_rm)
self.adj[v_rm].add(u_rm)
self.deg[u_rm] += 1
self.deg[v_rm] += 1

# Effectuer la transition propre
# 1. Retirer arete u_rm, v_rm avec marquage death
self._remove_edge_tracked(idx, current_step)
self.deg[u_rm] -= 1
self.deg[v_rm] -= 1

# 2. Ajouter arete u_add, v_add avec marquage birth
self._add_edge_tracked(u_add, v_add, current_step)
self.deg[u_add] += 1
self.deg[v_add] += 1

# 3. Mettre a jour les compteurs
delta_tri = dt_add - dt_rm
delta_tetra = dtetra_add - dtetra_rm
self.n_tri = new_n_tri
self.n_tetra += delta_tetra
self.S_curr = new_S

# 4. Classifier la transition (vocabulaire Rudolphe)

```

```

# alpha : reorganisation conservative
# beta : ascension triangulaire pure
# gamma : ascension tetraedrique
# delta : regression couplee tri+tetra
# epsilon : regression tetraedrique seule
# zeta : regression triangulaire seule
# eta : transition mixte (rare)
if delta_tri == 0 and delta_tetra == 0:
    ttype = 'alpha' # reorganisation
elif delta_tri > 0 and delta_tetra == 0:
    ttype = 'beta' # ascension Tri
elif delta_tri > 0 and delta_tetra > 0:
    ttype = 'gamma' # ascension Tetra
elif delta_tri < 0 and delta_tetra < 0:
    ttype = 'delta' # regression couplee
elif delta_tri == 0 and delta_tetra < 0:
    ttype = 'epsilon' # regression Tetra seule
elif delta_tri < 0 and delta_tetra == 0:
    ttype = 'zeta' # regression Tri seule
else:
    ttype = 'eta' # mixte (rare)
self.transitions[ttype] += 1

return True, ttype

def step_null(self, current_step):
    """Pas Null pour reference."""
    idx = self.rng.integers(len(self.edge_list))
    u_rm, v_rm = self.edge_list[idx]
    u_add = int(self.rng.integers(self.N))
    v_add = int(self.rng.integers(self.N))
    if u_add == v_add or v_add in self.adj[u_add]:
        return False, None
    u_add, v_add = min(u_add, v_add), max(u_add, v_add)
    if (u_add, v_add) == (u_rm, v_rm):
        return False, None

    dt_rm = len(self.adj[u_rm] & self.adj[v_rm])

    self.adj[u_rm].discard(v_rm)
    self.adj[v_rm].discard(u_rm)
    self.deg[u_rm] -= 1
    self.deg[v_rm] -= 1

    dt_add = len(self.adj[u_add] & self.adj[v_add])
    new_n_tri = self.n_tri - dt_rm + dt_add

```

```

        if new_n_tri < 0:
            self.adj[u_rm].add(v_rm)
            self.adj[v_rm].add(u_rm)
            self.deg[u_rm] += 1
            self.deg[v_rm] += 1
            return False, None

        # Acceptation auto en null
        self._remove_edge_tracked(idx, current_step)
        self._add_edge_tracked(u_add, v_add, current_step)
        self.adj[u_add].add(v_add)
        self.adj[v_add].add(u_add)
        self.deg[u_add] += 1
        self.deg[v_add] += 1
        self.n_tri = new_n_tri
        self.S_curr = self._compute_S(new_n_tri)

    return True, None

def update_engagement_profiles(self, current_step):
    """Pour chaque arete vivante, enregistre son engagement actuel."""
    # Sample : pour chaque arete vivante, calcul n_tri commun
    # Cout : O(|E| * deg). A appeler peu souvent.
    for idx, (u, v) in enumerate(self.edge_list):
        eid = self.edge_position_to_id[idx]
        n_common = len(self.adj[u] & self.adj[v])
        self.edge_engagement_profile[eid].append((current_step, n_common))
        if n_common > self.edge_max_engagement[eid]:
            self.edge_max_engagement[eid] = n_common

# =====
# RUN AVEC MESURES HIERARCHIQUES
# =====
def run_F(N, seed, total_steps, is_null, label='F'):
    t_start = time.time()
    type_label = 'NULL' if is_null else 'BLOB'
    log_id = ' [' + label + ' ' + type_label + ' N=' + str(N) + ' s=' + str(seed)
    print(log_id + ' Init ' + str(total_steps) + ' steps...', flush=True)

    sim = BlobGraphHier(N=N, seed=seed, is_null=is_null)
    step_func = sim.step_null if is_null else sim.step_blob

    print(log_id + ' Init OK |E|=' + str(len(sim.edge_list))
          + ' n_tri=' + str(sim.n_tri)
          + ' n_tetra=' + str(sim.n_tetra)
          + ' init_t=' + '{:.0f}'.format(time.time() - t_start) + 's', flush=True)

```

```

# Echantillonnage trajectoire (n_tri, n_tetra)
n_samples = 100
sample_steps = sorted(set([
    max(1, int(np.exp(np.log(total_steps) * i / n_samples)))
    for i in range(1, n_samples + 1)
]))

# Echantillonnage profils engagement (cout O(|E|), donc plus rare)
engagement_sample_count = 20
engagement_steps = sorted(set([
    max(1, int(total_steps * i / engagement_sample_count))
    for i in range(1, engagement_sample_count + 1)
]))

trajectory = [] # liste (step, n_tri, n_tetra, S, viab_total)
next_traj_idx = 0
next_eng_idx = 0
accepts = 0
transitions_per_sample = []

for i in range(total_steps):
    accepted, ttype = step_func(i + 1)
    if accepted:
        accepts += 1

# Sample trajectoire
if next_traj_idx < len(sample_steps) and (i + 1) >= sample_steps[next_traj_idx]:
    mu_deg = float(sim.deg.mean())
    D_curr = float(sim.deg.std()) / mu_deg if mu_deg > 0 else 0
    viab = sim.S_curr * sim.N
    trajectory.append({
        'step': i + 1,
        'n_tri': sim.n_tri,
        'n_tetra': sim.n_tetra,
        'S': sim.S_curr,
        'viab_total': viab,
        'D': D_curr,
        'acc_cumul': accepts / (i + 1),
        'transitions_snapshot': dict(sim.transitions)
    })
    next_traj_idx += 1
    if next_traj_idx % 20 == 0:
        print('    step ' + str(i+1) + '/' + str(total_steps)
              + ' n_tri=' + str(sim.n_tri)
              + ' n_tetra=' + str(sim.n_tetra)
              + ' acc=' + '{:.3f}'.format(accepts/(i+1)))

```

```

        + ' t=' + '{:.0f}'.format(time.time()-t_start) + 's', flush

# Sample profils engagement (cher)
if next_eng_idx < len(engagement_steps) and (i + 1) >= engagement_steps[n
    sim.update_engagement_profiles(i + 1)
    next_eng_idx += 1

total_time = time.time() - t_start
final_acc = accepts / total_steps if total_steps > 0 else 0

# =====
# ANALYSES F1, F2, F3
# =====

# F1 - Conservation simplexiiale verifiee par construction
f1_simplexial_conservation = (sim.deg.sum() == 2 * sim.E_target)

# F1 - Conservation viabilite : drift sur viab_total
if len(trajectory) >= 4:
    viab_first_q = np.mean([t['viab_total'] for t in trajectory[:len(trajecto
    viab_last_q = np.mean([t['viab_total'] for t in trajectory[3*len(trajecto
    viab_drift = (viab_last_q - viab_first_q) / abs(viab_first_q) if viab_fir
else:
    viab_drift = 0

# F2 - Distribution des durees de vie
edge_lifetimes_dead = [] # aretes mortes pendant la simulation
edge_max_engagement_dead = []
edge_engagement_categories_dead = {'isolee': 0, 'engagee_tri': 0, 'engagee_te

for eid, death in sim.edge_death.items():
    birth = sim.edge_birth.get(eid, 0)
    lifetime = death - birth
    edge_lifetimes_dead.append(lifetime)
    max_eng = sim.edge_max_engagement.get(eid, 0)
    edge_max_engagement_dead.append(max_eng)
    if max_eng == 0:
        edge_engagement_categories_dead['isolee'] += 1
    else:
        # Doit-on classer comme tetra ? On regarde si l arete est dans tetra
        # Approximation via max_eng : si max_eng >= 2, possiblement dans tetr
        # Plus precisement, il faudrait avoir trace l engagement tetra aussi.
        # Pour l instant : engagee_tri si max_eng >= 1
        if max_eng >= 2:
            edge_engagement_categories_dead['engagee_tetra'] += 1
        else:
            edge_engagement_categories_dead['engagee_tri'] += 1

```

```

# Aretes encore vivantes a la fin
edge_alive_count = len(sim.edge_list)

# Stats lifetimes
if edge_lifetimes_dead:
    lifetime_mean = float(np.mean(edge_lifetimes_dead))
    lifetime_median = float(np.median(edge_lifetimes_dead))
    lifetime_max = int(np.max(edge_lifetimes_dead))
else:
    lifetime_mean = 0
    lifetime_median = 0
    lifetime_max = 0

# Lifetime stratifie par engagement
by_cat_lifetimes = {'isolee': [], 'engagee_tri': [], 'engagee_tetra': []}
for i, eid in enumerate(sim.edge_death.keys()):
    max_eng = edge_max_engagement_dead[i]
    lifetime = edge_lifetimes_dead[i]
    if max_eng == 0:
        by_cat_lifetimes['isolee'].append(lifetime)
    elif max_eng >= 2:
        by_cat_lifetimes['engagee_tetra'].append(lifetime)
    else:
        by_cat_lifetimes['engagee_tri'].append(lifetime)

lifetime_by_cat_mean = {}
for cat, lst in by_cat_lifetimes.items():
    lifetime_by_cat_mean[cat] = float(np.mean(lst)) if lst else None

# F3 - Comptage final des transitions
transitions_final = dict(sim.transitions)
total_transitions = sum(transitions_final.values())

# Calcul flux ascendant vs descendant
ascensions = transitions_final.get('beta', 0) + transitions_final.get('gamma', 0)
regressions = (transitions_final.get('delta', 0)
                + transitions_final.get('epsilon', 0)
                + transitions_final.get('zeta', 0))
reorganisations = transitions_final.get('alpha', 0)
asymmetry = (ascensions - regressions) / (ascensions + regressions) if (ascen

# Sauvegarde JSON
save_path = OUTPUT_DIR + '/F_' + label + '_' + type_label + '_N' + str(N) + '
out = {
    'config': {
        'N': N, 'seed': seed, 'type': type_label, 'total_steps': total_steps,

```

```

        'kappa_0': KAPPA_0, 'beta': BETA, 'tau': TAU
    },
    'final': {
        'n_tri_final': int(sim.n_tri),
        'n_tetra_final': int(sim.n_tetra),
        'S_final': float(sim.S_curr),
        'acc_global': float(final_acc),
        'time_sec': float(total_time)
    },
    'F1_conservation': {
        'simplexial_conserved': bool(f1_simplexial_conservation),
        'viab_drift_relative': float(viab_drift)
    },
    'F2_aretes': {
        'edges_dead_total': len(sim.edge_death),
        'edges_alive_at_end': edge_alive_count,
        'lifetime_mean_steps': lifetime_mean,
        'lifetime_median_steps': lifetime_median,
        'lifetime_max_steps': lifetime_max,
        'categories_count': edge_engagement_categories_dead,
        'lifetime_by_category_mean': lifetime_by_cat_mean
    },
    'F3_transitions': {
        'counts': transitions_final,
        'total': total_transitions,
        'ascensions': ascensions,
        'regressions': regressions,
        'reorganisations': reorganisations,
        'asymmetry_ratio': float(asymmetry)
    },
    'trajectory': trajectory
}

try:
    with open(save_path, 'w') as f:
        json.dump(out, f)
except Exception as e:
    print(log_id + ' WARN save failed: ' + str(e))

print(log_id + ' DONE.')
print('    n_tri_final=' + str(sim.n_tri)
      + ' n_tetra_final=' + str(sim.n_tetra)
      + ' acc=' + '{:.3f}'.format(final_acc)
      + ' t=' + '{:.0f}'.format(total_time) + 's', flush=True)
print('    F1 simplex_conserve=' + str(f1_simplexial_conservation)
      + ' viab_drift=' + '{:+.3%}'.format(viab_drift), flush=True)
print('    F2 aretes mortes=' + str(len(sim.edge_death))
      + ' vivantes_fin=' + str(edge_alive_count)

```

```

        + ' lifetime_mean=' + '{:.0f}'.format(lifetime_mean), flush=True)
if not is_null:
    print('      F2 lifetime par categorie :', flush=True)
    for cat, lt in lifetime_by_cat_mean.items():
        print('          ' + cat + ' : ' + '{:.0f}'.format(lt) if lt else 'N/A')
    print('      F3 transitions :', flush=True)
    for ttype in ['alpha', 'beta', 'gamma', 'delta', 'epsilon', 'zeta', 'eta']:
        cnt = transitions_final.get(ttype, 0)
        pct = 100 * cnt / total_transitions if total_transitions > 0 else 0
        label_t = {
            'alpha': 'reorganisation',
            'beta': 'ascension Tri',
            'gamma': 'ascension Tetra',
            'delta': 'regression couplee',
            'epsilon': 'regression Tetra',
            'zeta': 'regression Tri',
            'eta': 'mixte'
        }[ttype]
        print('          ' + ttype + ' (' + label_t + ') : ' + str(cnt)
              + ' ({:.1f}%}'.format(pct), flush=True)
    print('      F3 asymetrie ascension/regression=' + '{:+.3f}'.format(asymmet

del sim
gc.collect()
return out

# =====
# MAIN PROTOCOLE F
# =====
print('')
print('=' * 75)
print('  PROTOCOLE F : Mesures hierarchiques')
print('=' * 75)
print('  Cadre : conservation simplexiale, vocabulaire ascension/regression')
print('  N=50000, 30N pas (1.5M), 2 seeds Blob + 1 seed Null')
print('=' * 75)

N_FIXED = 50000
TOTAL_STEPS = 30 * N_FIXED # 1.5M

RUNS = [
    {'N': N_FIXED, 'seed': 0, 'is_null': False, 'label': 'F.30N'},
    {'N': N_FIXED, 'seed': 1, 'is_null': False, 'label': 'F.30N'},
    {'N': N_FIXED, 'seed': 100, 'is_null': True, 'label': 'F.30N'},
]

```

```

print('')
print(' Configuration :')
print('      N=' + str(N_FIXED) + ', steps=' + str(TOTAL_STEPS) + ' (30N)')
print('      Total runs : ' + str(len(RUNS)))
print('      Duree estimee par run : 10-20 min (overhead tetraedres)')
print('      Duree totale estimee : 35-60 min')
print('=' * 75)

t_global = time.time()
all_results = []

for run_cfg in RUNS:
    print('')
    try:
        r = run_F(run_cfg['N'], run_cfg['seed'], TOTAL_STEPS,
                  is_null=run_cfg['is_null'], label=run_cfg['label'])
        all_results.append(r)
    except Exception as e:
        print(' ERREUR run : ' + str(e))
        import traceback
        traceback.print_exc()

global_time = time.time() - t_global

# =====
# RAPPORT FINAL CONSOLIDE
# =====
print('')
print('=' * 75)
print(' RAPPORT PROTOCOLE F')
print('=' * 75)
print(' Temps total : ' + '{:.1f}'.format(global_time / 60) + ' min')

# F1 Conservation
print('')
print(' --- F1 : Conservation ---')
print(' ' + 'run'.ljust(15) + 'simplex_OK'.ljust(14) + 'viab_drift')
for r in all_results:
    cfg = r['config']
    label_run = cfg['type'] + ' s=' + str(cfg['seed'])
    simp = r['F1_conservation']['simplexial_conserved']
    drift = r['F1_conservation']['viab_drift_relative']
    print(' ' + label_run.ljust(15) + str(simp).ljust(14) + '{:+.3%}'.format(dri

# F2 Cycle de vie
print('')

```

```

print(' --- F2 : Cycle de vie des aretes ---')
print(' ' + 'run'.ljust(15) + 'lifetime_mean'.ljust(16) + 'isolee'.ljust(10) + '
for r in all_results:
    cfg = r['config']
    label_run = cfg['type'] + ' s=' + str(cfg['seed'])
    lt = r['F2_aretes']['lifetime_mean_steps']
    cats = r['F2_aretes']['lifetime_by_category_mean']
    print(' ' + label_run.ljust(15)
          + '{:.0f}'.format(lt).ljust(16)
          + ('{:.0f}'.format(cats['isolee']) if cats['isolee'] else 'N/A').ljust(
          + ('{:.0f}'.format(cats['engagee_tri']) if cats['engagee_tri'] else 'N/
          + ('{:.0f}'.format(cats['engagee_tetra']) if cats['engagee_tetra'] else

# Verdict stabilisation hierarchique
blob_results = [r for r in all_results if r['config']['type'] == 'BLOB']
if blob_results:
    print('')
    print(' Test stabilisation hierarchique (Blob, moyenne sur seeds) :')
    cats_avg = defaultdict(list)
    for r in blob_results:
        for cat, v in r['F2_aretes']['lifetime_by_category_mean'].items():
            if v is not None:
                cats_avg[cat].append(v)
    means = {cat: np.mean(lst) for cat, lst in cats_avg.items() if lst}
    if 'isolee' in means and ('engagee_tri' in means or 'engagee_tetra' in means)
        iso = means['isolee']
        eng_t = means.get('engagee_tri', 0)
        eng_T = means.get('engagee_tetra', 0)
        max_eng = max(eng_t, eng_T)
        if max_eng > 0 and iso > 0:
            ratio = max_eng / iso
            print(' Ratio lifetime engagee / isolee : {:.2f}x'.format(ratio))
            if ratio > 1.5:
                print(' -> Stabilisation hierarchique CONFIRMEE [M]')
            elif ratio > 1.1:
                print(' -> Effet faible mais detectable')
            elif ratio > 0.9:
                print(' -> Pas d effet de stabilisation')
            else:
                print(' -> Effet inverse : aretes engagees vivent moins longte

# F3 Transitions
print('')
print(' --- F3 : Transitions hierarchiques (Blob seulement) ---')
for r in blob_results:
    cfg = r['config']
    print(' Seed ' + str(cfg['seed']) + ' :')

```

```

counts = r['F3_transitions']['counts']
total_t = r['F3_transitions']['total']
asy = r['F3_transitions']['asymmetry_ratio']
print('    Total transitions : ' + str(total_t))
for ttype in ['alpha', 'beta', 'gamma', 'delta', 'epsilon', 'zeta', 'eta']:
    cnt = counts.get(ttype, 0)
    pct = 100 * cnt / total_t if total_t > 0 else 0
    label_t = {
        'alpha': 'reorganisation',
        'beta': 'ascension Tri',
        'gamma': 'ascension Tetra',
        'delta': 'regression couplee',
        'epsilon': 'regression Tetra',
        'zeta': 'regression Tri',
        'eta': 'mixte'
    }[ttype]
    print('        ' + ttype + ' (' + label_t.ljust(20) + ') : ' + str(cnt) + '
print('    Asymetrie A-R / A+R = {:.3f}'.format(asy))

# Tetraedres : evolution
print('')
print(' --- F3 bis : Tetraedres ---')
for r in blob_results:
    cfg = r['config']
    n_tetra_final = r['final']['n_tetra_final']
    n_tri_final = r['final']['n_tri_final']
    ratio_r3 = n_tetra_final / n_tri_final if n_tri_final > 0 else 0
    print(' Seed ' + str(cfg['seed']) + ' : n_tri=' + str(n_tri_final)
          + ' n_tetra=' + str(n_tetra_final)
          + ' r3=n_tetra/n_tri={:.4f}'.format(ratio_r3))

# Comparaison Blob vs Null
print('')
print(' --- Comparaison Blob/Null ---')
null_results = [r for r in all_results if r['config']['type'] == 'NULL']
if blob_results and null_results:
    blob_tri = np.mean([r['final']['n_tri_final'] for r in blob_results])
    blob_tetra = np.mean([r['final']['n_tetra_final'] for r in blob_results])
    null_tri = np.mean([r['final']['n_tri_final'] for r in null_results])
    null_tetra = np.mean([r['final']['n_tetra_final'] for r in null_results])
    print(' Blob : n_tri={:.0f}'.format(blob_tri) + ' n_tetra={:.0f}'.format(blob_tetra))
    print(' Null : n_tri={:.0f}'.format(null_tri) + ' n_tetra={:.0f}'.format(null_tetra))
    if null_tri > 0:
        print(' Ratio Tri Blob/Null : {:.2f}x'.format(blob_tri / null_tri))
    if null_tetra > 0:
        print(' Ratio Tetra Blob/Null : {:.2f}x'.format(blob_tetra / null_tetra))

```

```
print('')
print('  Fichiers JSON sauvegardes :')
for f in sorted(os.listdir(OUTPUT_DIR)):
    print('    ' + OUTPUT_DIR + '/' + f)

print('')
print('=' * 75)
print('  FIN PROTOCOLE F')
print('=' * 75)
```